

Unit-4

Introduction to Digital Electronics



Dr. Jyoti Prasanna Patra
Assistant Professor, ECE, CGU
PhD, NIT Rourkela

Boolean Algebra

Boolean algebra is a branch of mathematics dealing with variables that have only two values: True (1) or False (0).

- Introduced by George Boole in 1854
- Three basic operations: **AND**, **OR**, and **NOT**
- The **AND** operator is denoted by a dot ($\cdot$),

$x \cdot y$ or xy is read: x **AND** y

- The **OR** operator is denoted by a plus (+)

$x + y$ is read: x **OR** y

- The **NOT** operator is denoted by (') or an overbar ($\bar{}$).

x' or $\bar{x}$ is the complement of x

$x \cdot y$ is the **AND** operator

$x + y$ is the **OR** operator

x' is the **NOT** operator

AND ($\cdot$)

$$0 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$

$$1 \cdot 0 = 0$$

$$1 \cdot 1 = 1$$

Multiplication and
AND give identical
results

OR (+)

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1$$

Addition and **OR**
give different
Results for 1+1

NOT ($\bar{}$)

$$\bar{0} = 1$$

$$\bar{1} = 0$$

No corresponding
Math operator for
NOT

Importance of Boolean Algebra

- A logic gate is **a device that performs a Boolean function**, a logical operation performed on one or more binary inputs that produces a single binary output.
- These circuits use signals with two possible values
- Logic **0** is a **low** voltage signal (around 0 volts)
- Logic **1** is a **high** voltage signal (e.g. 5 or 3.3 volts)
- The physical value of a signal is the actual voltage it carries, while its logic value is either 0 (low) or 1 (high)
- Having only two logic values (0 and 1) simplifies the implementation of the digital circuit

Basic Logic Function

Input/Output Definitions

- Input:
 - logic 1 is switch closed
 - logic 0 is switch open
- Output:
 - logic 1 is lamp on
 - logic 0 is lamp off.

AND (.)

$$0 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$

$$1 \cdot 0 = 0$$

$$1 \cdot 1 = 1$$

OR (+)

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1$$

NOT ($\bar{}$)

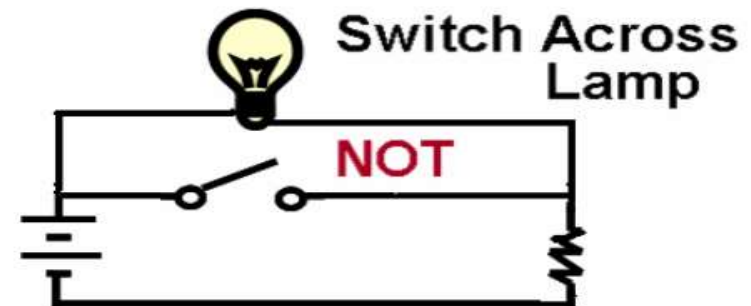
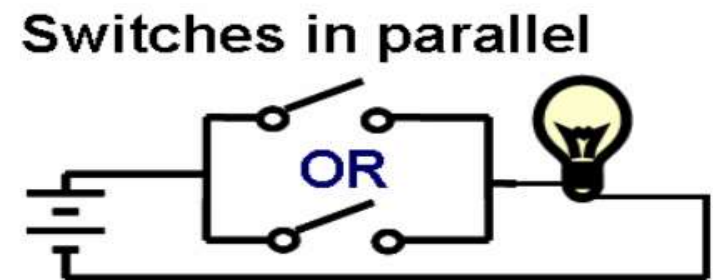
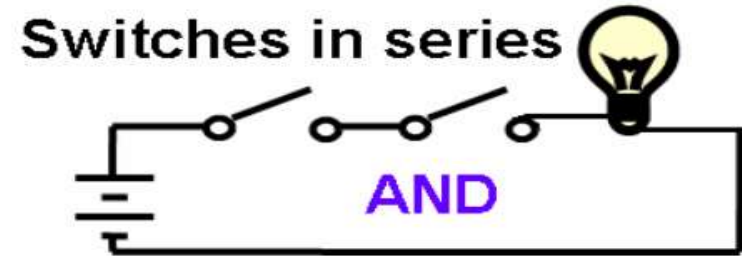
$$\bar{0} = 1$$

$$\bar{1} = 0$$

Multiplication and
AND give identical
results

Addition and **OR**
give different
Results for 1+1

No corresponding
Math operator for
NOT

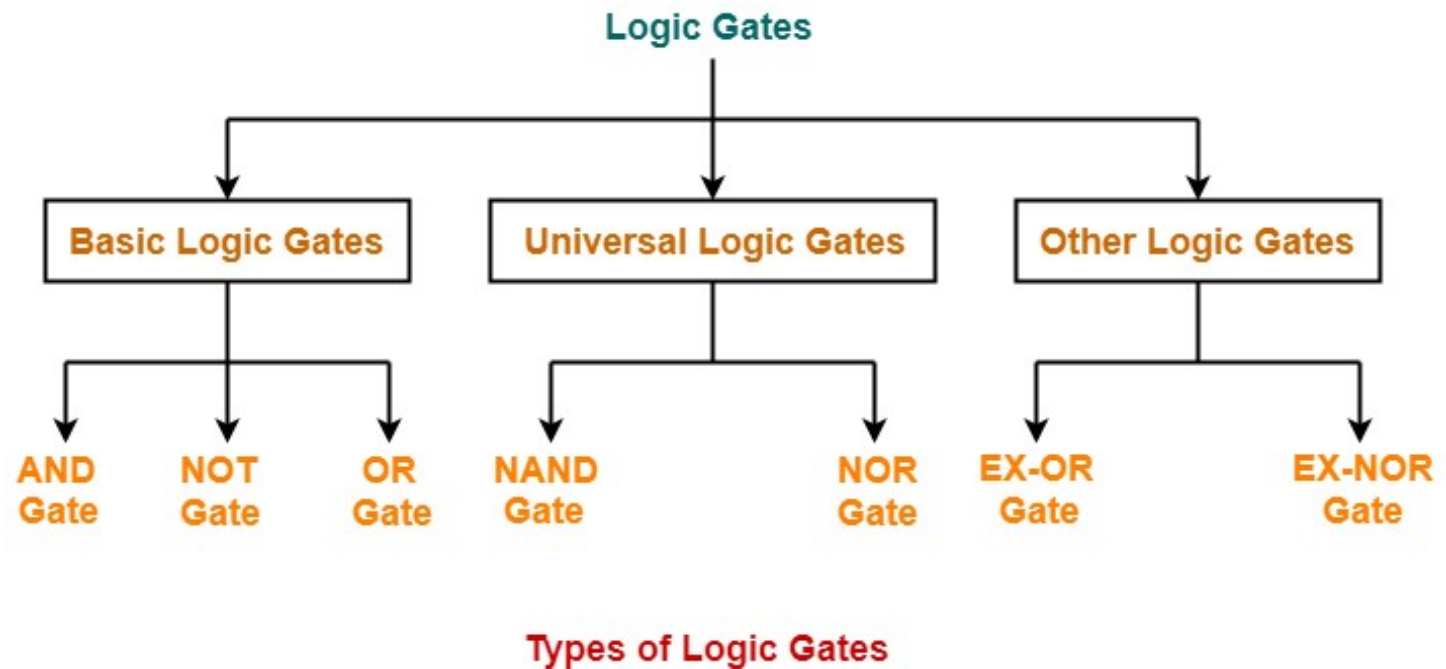


Logic Gates

- A **gate** is a device that performs a basic operation on electrical signals
- Gates are combined into **circuits** to perform more complicated tasks
- describing the behavior of gates and circuits by
 - Boolean expressions
 - logic diagrams
 - truth tables

Seven types of Gates

- ▶ NOT
- ▶ AND
- ▶ OR
- ▶ XOR
- ▶ XNOR
- ▶ NAND
- ▶ NOR



Logic Gates



OR Gate



AND Gate



NOT Gate



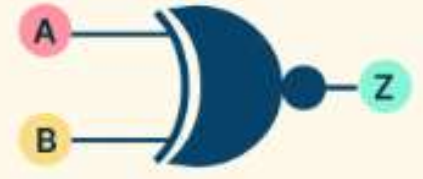
NAND Gate



NOR Gate



XOR Gate



XNOR Gate

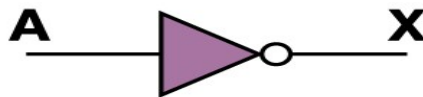
NOT Gate

- A NOT gate accepts one input value and produces one output value
- By definition, if the input value for a NOT gate is 0, the output value is 1, and if the input value is 1, the output is 0
- A NOT gate is sometimes referred to as an *inverter* because it inverts the input value

Boolean Expression

$$X = A'$$

Logic Diagram Symbol



Truth Table

A	X
0	1
1	0

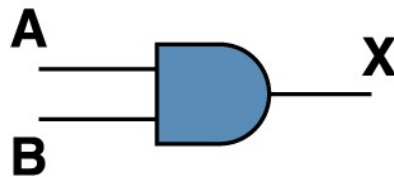
AND Gate

- An AND gate accepts two input signals
- If the two input values for an AND gate are both 1, the output is 1; otherwise, the output is 0

Boolean Expression

$$X = A \cdot B$$

Logic Diagram Symbol



Truth Table

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

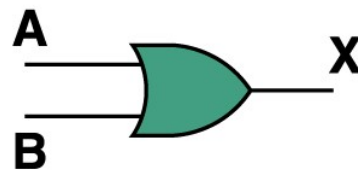
OR Gate

- If the two input values are both 0, the output value is 0; otherwise, the output is 1

Boolean Expression

$$X = A + B$$

Logic Diagram Symbol



Truth Table

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

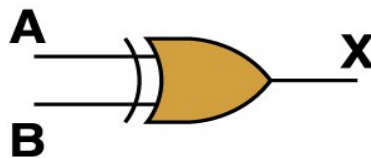
XOR Gate

- XOR, or *exclusive* OR, gate
 - An XOR gate produces 0 if its two inputs are the same, and a 1 otherwise
 - Note the difference between the XOR gate and the OR gate; they differ only in one input situation
 - When both input signals are 1, the OR gate produces a 1 and the XOR produces a 0

Boolean Expression

$$X = A \oplus B$$

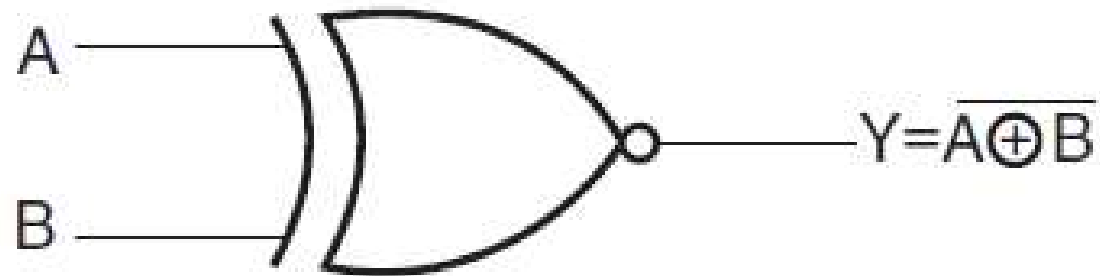
Logic Diagram Symbol



Truth Table

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate



$$Y = \overline{(A \oplus B)} = (A.B + \overline{A}.\overline{B})$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

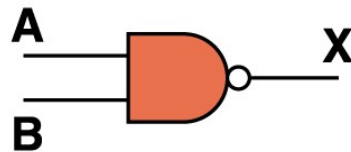
NAND and NOR Gates

- The NAND and NOR gates are essentially the opposite of the AND and OR gates, respectively

Boolean Expression

$$X = (A \cdot B)'$$

Logic Diagram Symbol



Truth Table

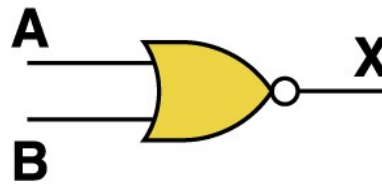
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

Boolean Expression

$$X = (A + B)'$$

Logic Diagram Symbol



Truth Table

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

Review of Gate Processing

- A NOT gate inverts its single input value
- An AND gate produces 1 if both input values are 1
- An OR gate produces 1 if one or the other or both input values are 1
- An XOR gate produces 1 if one or the other (but not both) input values are 1
- A NAND gate produces the opposite results of an AND gate
- A NOR gate produces the opposite results of an OR gate

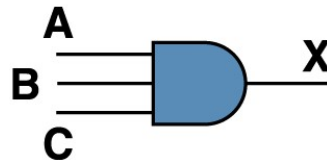
Gates with More Inputs

- Gates can be designed to accept three or more input values
- A three-input AND gate, for example, produces an output of 1 only if all input values are 1

Boolean Expression

$$X = A \cdot B \cdot C$$

Logic Diagram Symbol



Truth Table

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Properties of Boolean Algebra

1. Closure: the result of any Boolean operation is in $B = \{0, 1\}$
2. Identity element with respect to $+$ is 0: $x + 0 = 0 + x = x$

Identity element with respect to $\cdot$ is 1: $x \cdot 1 = 1 \cdot x = x$

3. Commutative with respect to $+$: $x + y = y + x$

Commutative with respect to $\cdot$: $x \cdot y = y \cdot x$

4. $\cdot$ is distributive over $+$: $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$

$+$ is distributive over $\cdot$: $x + (y \cdot z) = (x + y) \cdot (x + z)$

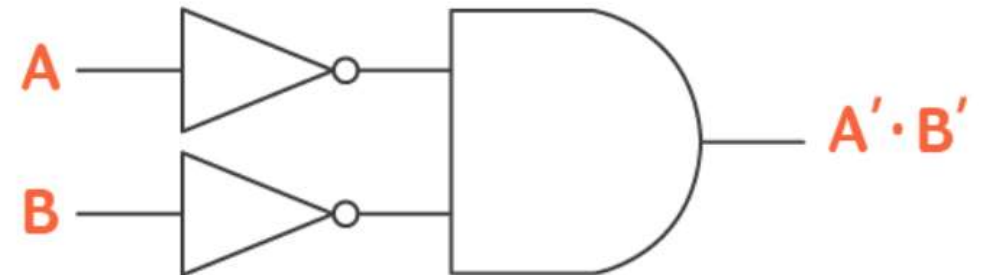
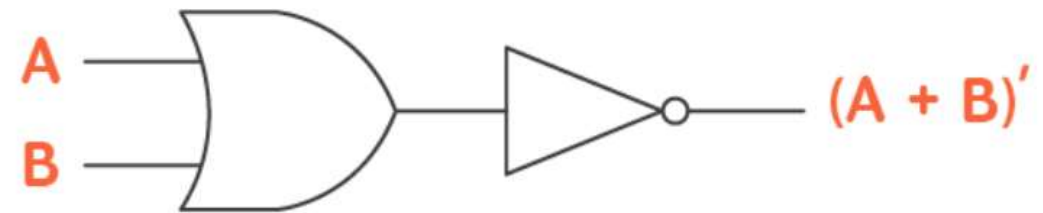
5. For every x in B , there exists x' in B (called complement of x) such that: $x + x' = 1$ and $x \cdot x' = 0$

Properties of Boolean Algebra

	Property	Dual Property
Identity	$x + 0 = x$	$x \cdot 1 = x$
Complement	$x + x' = 1$	$x \cdot x' = 0$
Null	$x + 1 = 1$	$x \cdot 0 = 0$
Idempotence	$x + x = x$	$x \cdot x = x$
Involution	$(x')' = x$	
Commutative	$x + y = y + x$	$xy = yx$
Associative	$(x + y) + z = x + (y + z)$	$(xy)z = x(yz)$
Distributive	$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$
Absorption	$x + xy = x$	$x(x + y) = x$
Simplification	$x + x'y = x + y$	$x(x' + y) = xy$
De Morgan	$(x + y)' = x' y'$	$(xy)' = x' + y'$

De Morgan's Law

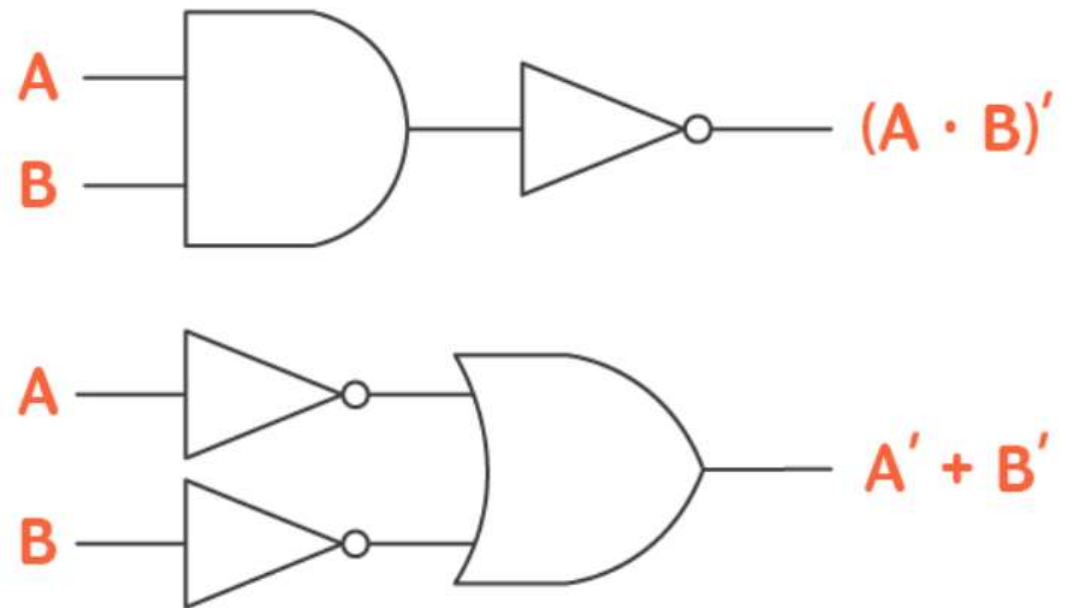
Inputs			Outputs			
A	B	A + B	$(A + B)'$	A'	B'	A · B
0	0	0	1	1	1	1
0	1	1	0	0	1	0
1	0	1	0	1	0	0
1	1	1	0	0	0	0



$$(A + B)' = A' \cdot B'$$

De Morgan's Law

Inputs			Outputs			
A	B	$A \cdot B$	$(A \cdot B)'$	A'	B'	$A' + B'$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0



$$(A \cdot B)' = A' + B'$$

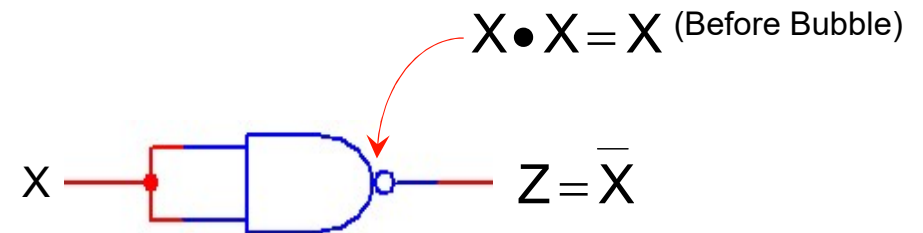
Universal Gate – NAND and NOR

- The **NAND** and NOR gates are self-sufficient (can built any logic circuits).
- Therefore, NAND and NOR are complete set of logics
- The **NAND** and NOR gate can be used to replace an **AND** gate, an **OR** gate, or an **INVERTER** gate.

NOT gate using NAND Gate



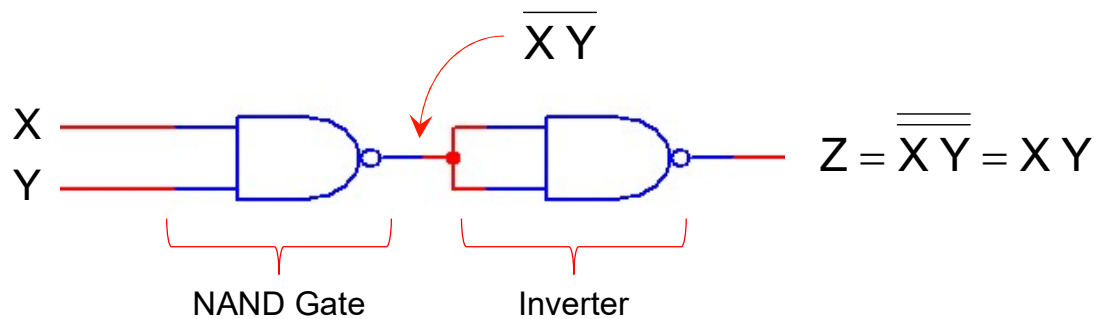
X	Y	Z
0	0	1
0	1	1
1	0	1
1	1	0



x	z
0	1
1	0

} Equivalent to Inverter

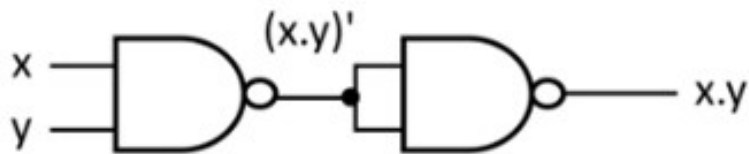
NAND Gate as an AND Gate



X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

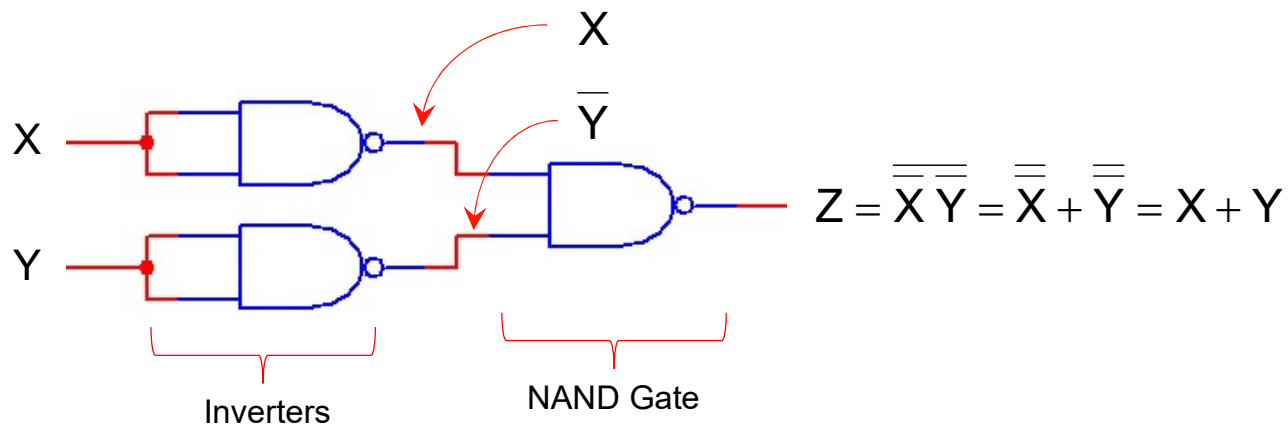
Equivalent to AND Gate

- Implementing AND using NAND gates:



$$((x.y)'(x.y)')' = ((x.y)')' \text{ idempotency} \\ = (xy) \text{ involution}$$

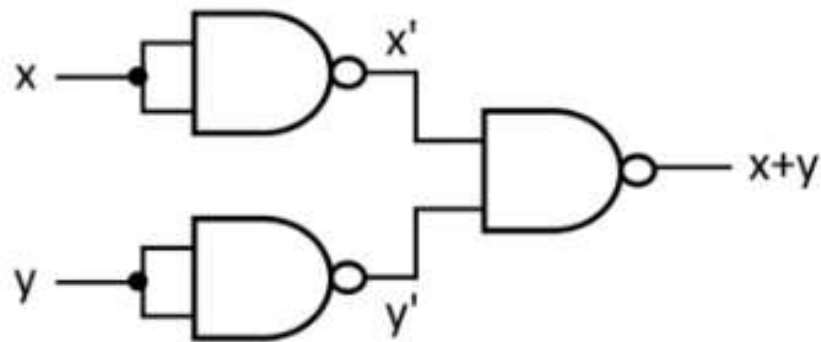
NAND Gate as an OR Gate



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

Equivalent to OR Gate

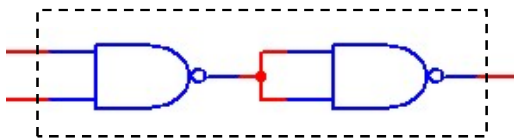
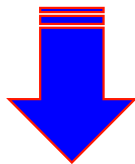
■ Implementing OR using NAND gates:



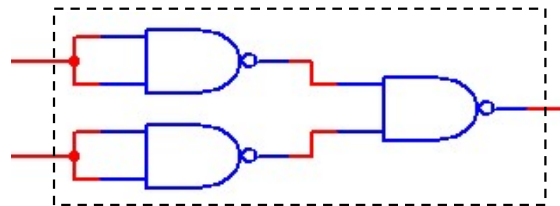
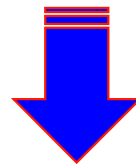
$$\begin{aligned}
 ((x.x)'(y.y'))' &= (x'.y')' && \text{idempotency} \\
 &= x''+y'' && \text{DeMorgan} \\
 &= x+y && \text{involution}
 \end{aligned}$$

NAND Gate Equivalent to AOI Gates

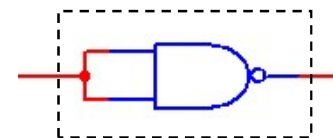
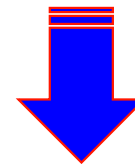
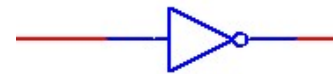
AND



OR



INVERTER

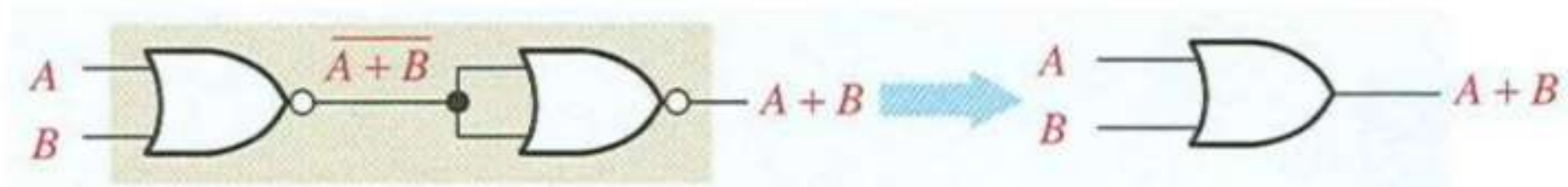


- NOR Gate as an Inverter

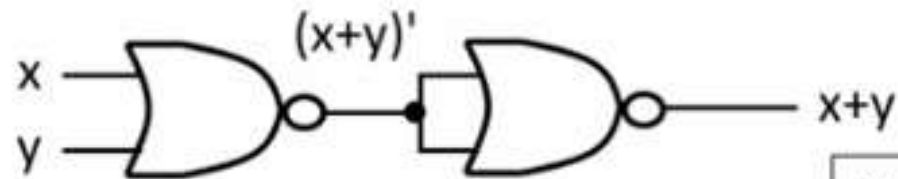


$$(x+x)' = x' \quad (\text{T1: idempotency})$$

- Two NOR Gates as an OR Gate

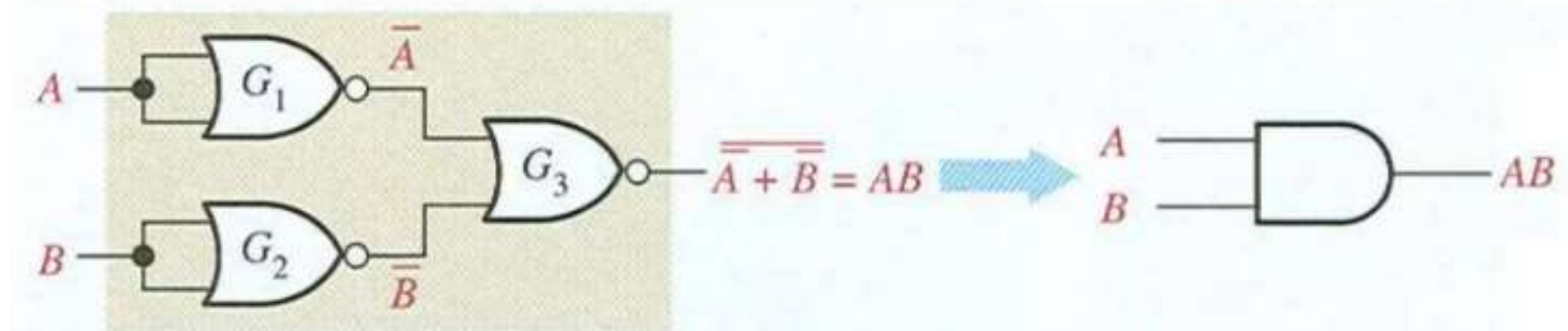


- Implementing OR using NOR gates:

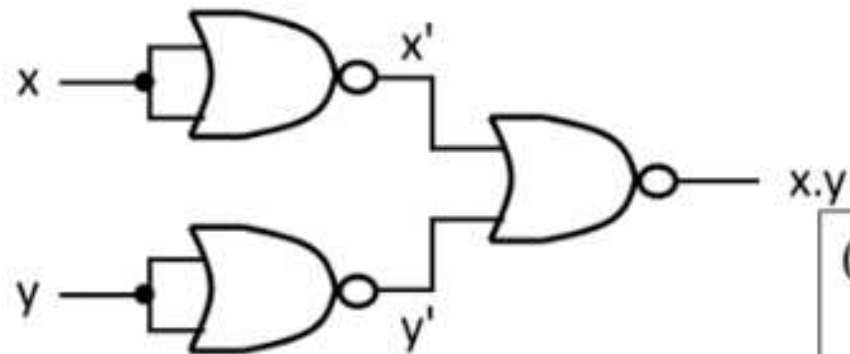


$$\begin{aligned} ((x+y)' + (x+y)')' &= ((x+y)')' && \text{idempotency} \\ &= (x+y) && \text{involution} \end{aligned}$$

- Three NOR Gates as an AND Gate



- Implementing AND using NOR gates:



$((x+x)' + (y+y'))' = (x' + y')'$	idempotency
$= x'' . y''$	DeMorgan
$= x.y$	involution

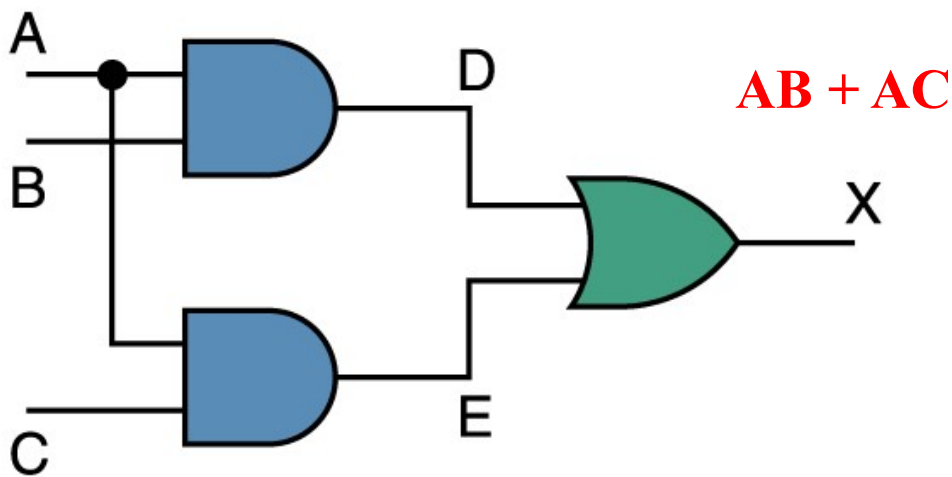
Circuits

- Two general categories
 - In a **combinational circuit**, the input values explicitly determine the output
 - In a **sequential circuit**, the output is a function of the input values as well as the existing state of the circuit
- As with gates, we can describe the operations of entire circuits using three notations
 - Boolean expressions
 - logic diagrams
 - truth tables

Combinational Circuits

Gates are combined into circuits by using the output of one gate as the input for another.

- ▶ Because there are three inputs to this circuit, eight rows are required to describe all possible input combinations
- ▶ This same circuit using Boolean algebra: $(AB + AC)$



A	B	C	D	E	X
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Half-Adders

- At the digital logic level, addition is performed in binary
- Addition operations are carried out by special circuits called, appropriately, **adders**
- The result of adding two binary digits could produce a *carry value*
- Recall that $1 + 1 = 10$ in base two
- A circuit that computes the sum of two bits and produces the correct carry bit is called a **half adder**



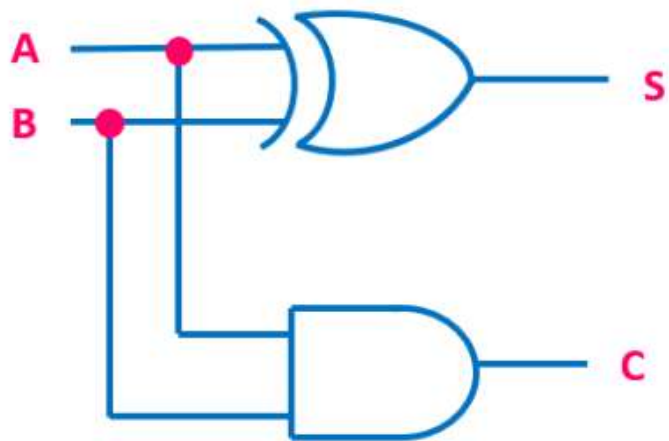
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half-Adders

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

Logic Circuit



- Circuit diagram representing a half adder

- Two Boolean expressions:

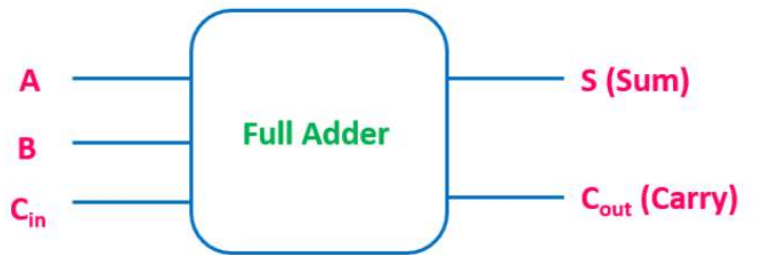
$$\text{sum} = A \oplus B$$

$$\text{carry} = AB$$

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full Adder

A full adder is a combinational circuit that forms the arithmetic sum of input; it consists of three inputs and two outputs.



$$\begin{aligned}
 S &= \overline{A} \overline{B} C_{in} + \overline{A} B \overline{C}_{in} + A \overline{B} \overline{C}_{in} + A B C_{in} \\
 &= \overline{A} (\overline{B} C_{in} + B \overline{C}_{in}) + A (\overline{B} \overline{C}_{in} + B C_{in}) \\
 &= \overline{A} (B \oplus C_{in}) + A (\overline{B \oplus C_{in}})
 \end{aligned}$$

$$S = A \oplus B \oplus C_{in}$$

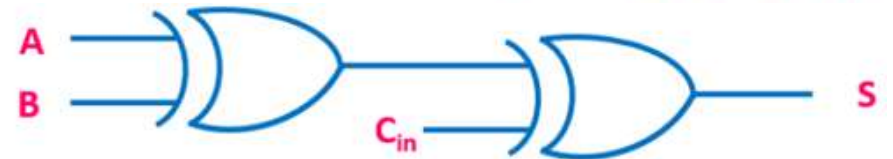
$$C_{out} = \overline{A} B C_{in} + A \overline{B} C_{in} + A B \overline{C}_{in} + A B C_{in}$$

A	B	C	Sum (S)	Carry (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

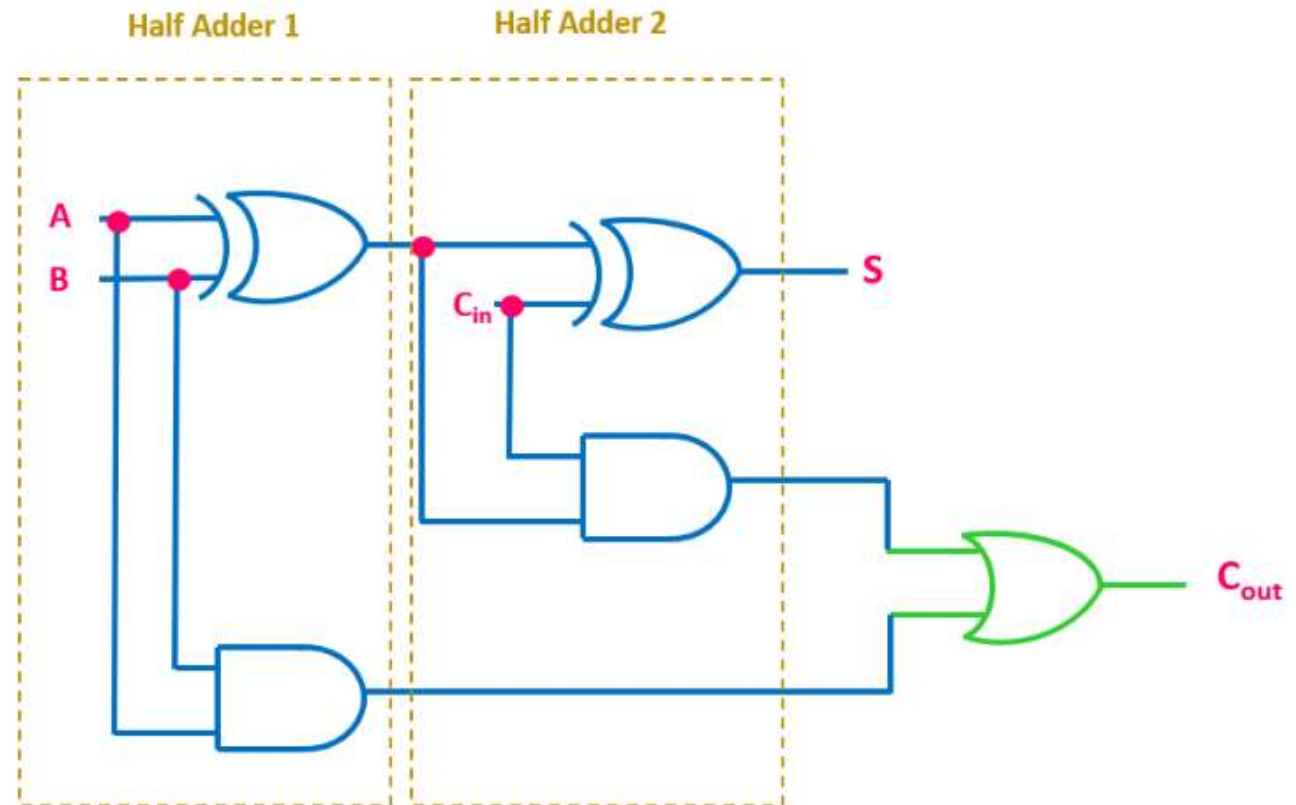
Full Adder

$$\begin{aligned}
 C_{out} &= \bar{A} B C_{in} + A \bar{B} C_{in} + A B \bar{C}_{in} + A B C_{in} \\
 &= A B (C_{in} + \bar{C}_{in}) + C_{in} (\bar{A} B + A \bar{B}) \\
 &= A B + C_{in} (A \oplus B)
 \end{aligned}$$

$$S = A \oplus B \oplus C_{in}$$



A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Half Subtractor

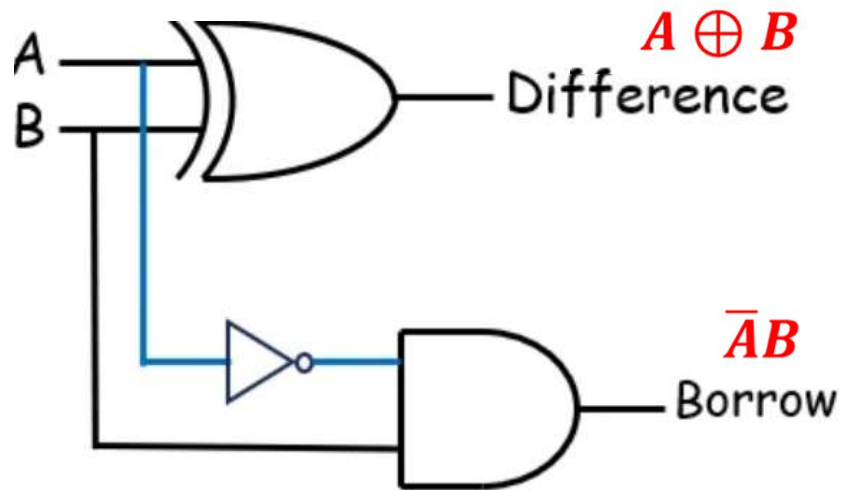
A Half Subtractor is a digital circuit designed to perform the subtraction of two single-bit binary numbers.

It produces two outputs: a difference bit (D) and a borrow bit (B).

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Components of a Half Subtractor

- **XOR Gate:** Computes the difference bit (D).
- **AND Gate:** Computes the borrow bit (B).



Half Subtractor Circuit Diagram

Logical expressions of these outputs are:

- Difference (D) = A XOR B
- Borrow (B) = A' AND B

Standard Forms of Boolean Expressions

Usually, literals (Boolean variables in complemented or uncomplemented form) terms are arranged in one of the two standard forms of switching equations :

- Sum of product form (SOP) and
- Product of sum form (POS)

Sum of product(SOP)

SOP form is a set of product (AND) terms that are summed (OR) together.

When an expression or term is represented in a sum of binary terms known as minterms or sum of products.

$$F = AB + A'C + BC$$

Product of sum(POS)

A technique of explaining a Boolean expression through a set of max terms or sum terms, is known as POS (product of sum).

$$F = (A + B)(A' + C)(B + C)$$

Standard vs Canonical Forms

Feature	Standard Form	Canonical Form
SOP	May have missing literals in product terms	Every product term contains ALL variables
POS	May have missing literals in sum terms	Every sum term contains ALL variables
Example	$F = AB + AC$ (Standard SOP)	$F = A'BC + AB'C + ABC' + ABC$ (Canonical SOP)

In **Digital Electronics**, the **canonical form** means writing a Boolean function so that **each term contains all variables exactly once** (either complemented or uncomplemented).

- **Minterms** = Canonical SOP form
- **Maxterms** = Canonical POS form

Variables			Minterms	Maxterms
A	B	C	m_i	M_i
0	0	0	$\bar{A} \bar{B} \bar{C} = m_0$	$A + B + C = M_0$
0	0	1	$\bar{A} \bar{B} C = m_1$	$A + B + \bar{C} = M_1$
0	1	0	$\bar{A} B \bar{C} = m_2$	$A + \bar{B} + C = M_2$
0	1	1	$\bar{A} B C = m_3$	$A + \bar{B} + \bar{C} = M_3$
1	0	0	$A \bar{B} \bar{C} = m_4$	$\bar{A} + B + C = M_4$
1	0	1	$A \bar{B} C = m_5$	$\bar{A} + B + \bar{C} = M_5$
1	1	0	$A B \bar{C} = m_6$	$\bar{A} + \bar{B} + C = M_6$
1	1	1	$A B C = m_7$	$\bar{A} + \bar{B} + \bar{C} = M_7$

Minterms and Maxterms for three variables

	SOP	POS
1	SOP stands for Sum of Products.	POS stands for Product of Sums.
2	It is a technique of defining the Boolean terms as the sum of product terms.	It is a technique of defining Boolean terms as a product of sum terms.
3	It prefers minterms.	It prefers maxterms.
4	In the case of SOP, the minterms are defined as 'm'.	In the case of POS, the Maxterms are defined as 'M'
5	It gives HIGH (1) output.	It gives LOW(0) output.
6	In SOP, we can get the final term by adding the product terms.	In POS, we can get the final term by multiplying the sum terms.

Convert the given expression in Canonical SOP form:

$$Y = AC + AB + BC$$

$$Y = AC + AB + BC$$

$$= AC (B + \bar{B}) + AB (C + \bar{C}) + BC (A + \bar{A})$$

$$[\because A + \bar{A} = 1; B + \bar{B} = 1; C + \bar{C} = 1]$$

$$= ABC + A\bar{B}C + ABC + AB\bar{C} + ABC + \bar{A}BC$$

$$= A\bar{B}C + AB\bar{C} + ABC + \bar{A}BC \quad \text{Rule : } [A + A = A]$$

Convert the given expression in Canonical SOP form:

$$Y = A + AB + ABC$$

$$Y = A + AB + ABC$$

$$= A \cdot (B + \bar{B}) \cdot (C + \bar{C}) + AB \cdot (C + \bar{C}) + ABC$$

$$= (AB + A\bar{B}) \cdot (C + \bar{C}) + ABC + AB\bar{C} + ABC$$

$$= ABC + AB\bar{C} + A\bar{B}C + A\bar{B}\bar{C} + ABC + AB\bar{C} + ABC$$

$$= ABC + AB\bar{C} + A\bar{B}C + A\bar{B}\bar{C} \quad \text{Rule : } [A + A = A]$$

Convert the given expression in Canonical POS form :

$$Y = (A + B) (B + C) (A + C)$$

$$Y = (A + B) (B + C) (A + C)$$

$$= (A + B + C \cdot \bar{C}) (B + C + A \cdot \bar{A}) (A + C + B \cdot \bar{B})$$

$$= (A + B + C)(A + B + \bar{C})(A + B + C)(\bar{A} + B + C)(A + B + C)(A + \bar{B} + C)$$

$$= (A + B + C) (A + B + \bar{C}) (\bar{A} + B + C) (A + \bar{B} + C)$$

Convert the given expression in Canonical POS form :

$$Y = A (A + B) (A + B + C)$$

$$Y = A (A + B) (A + B + C)$$

$$= (A + B + \bar{B} + C + \bar{C}) (A + B + C + \bar{C}) (A + B + C)$$

$$[\because B + \bar{B} = 1 ; C + \bar{C} = 1]$$

$$= (A + B + \bar{B} + C) (A + B + \bar{B} + C) (A + B + C) (A + B + \bar{C}) (A + B + C)$$

$$= (A + B + C) (A + \bar{B} + C) (A + B + \bar{C}) (A + \bar{B} + \bar{C})$$

$$(A + B + C) (A + B + \bar{C}) (A + B + C)$$

$$= (A + B + C) (A + \bar{B} + C) (A + B + \bar{C}) (A + \bar{B} + \bar{C})$$

Complementing Boolean Functions

- What is the complement of $f = x'yz' + xy'z'$?
- Use DeMorgan's Theorem:
 - Complement each variable and constant
 - Interchange AND and OR operators
- So, what is the complement of $f = x'yz' + xy'z'$?

Answer: $f' = (x + y' + z)(x' + y + z)$

- **Example 2:** Complement $g = (a' + bc)d' + e$
- **Answer:** $g' = (a(b' + c') + d)e'$

Algebraic Manipulation of Expressions

- The objective is to acquire skills in manipulating Boolean expressions, to transform them into simpler form.
- **Example 1:** prove $x + xy = x$ (absorption theorem)
- **Proof:**

$x + xy = x \cdot 1 + xy$	$x \cdot 1 = x$
$= x \cdot (1 + y)$	Distributive $\cdot$ over $+$
$= x \cdot 1 = x$	$(1 + y) = 1$
- **Example 2:** prove $x + x'y = x + y$ (simplification theorem)
- **Proof:**

$x + x'y = (x + x')(x + y)$	Distributive $+$ over $\cdot$
$= 1 \cdot (x + y)$	$(x + x') = 1$
$= x + y$	